Depth-First Search

Depth-First Search

- An algorithm for traversing or searching graph data structures that explores as far as possible along each branch before backtracking

BFS and DFS Comparison:

Feature	Breadth-First Search (BFS)	Depth-First Search (DFS)
Mechanism	Uses a **Queue** (FIFO) Explores level by level	Inherently recursive; uses a **Stack** (the built-in call stack) to remember where to backtrack
Output	A **Breadth-First Tree** starting from the source node	Typically iterates over all vertices, creating a **Depth-First Forest** (a collection of separate, unconnected trees)

The Timestamp System:

Instead of keeping track of the "distance" from a source node (like BFS), DFS tracks the exact "time" a vertex was interacted with, using a global `time` counter that increments every time a vertex changes colour

- **Discovery Time $d[v]$:** The `time` when vertex v is first found and coloured **Grey**
- **Finish Time $f[v]$:** The `time` when the algorithm has finished examining *all* of the outgoing edges from vertex v, and it is coloured **Black**
- **Rule:** For any vertex v, its discovery time is always less than its finish time $\left(d[v] < f[v]\right)$

Algorithm: Depth-First Search (DFS)

Because DFS creates a **forest** of multiple unconnected trees, the algorithm is split into two distinctive parts: a "Main" loop that ensures that every vertex is checked, and a recursive "Visit" function that explores the paths from each vertex

Outer Function (Main Loop):

1. **Set Up:** Set all vertices to **White** (undiscovered) and all predecessors π to `None`
2. **Reset Clock:** Set the global `time` counter to 0
3. **Forest Loop:** Iterate through every vertex u in the graph. If u is **White**, call the recursive exploration function `DFS_Visit(u)`

- *This loop guarantees that we build a Depth-First Search, even if the graph is disconnected*

Inner Function (`DFS_Visit(u)` - Recursive Explorer Function):

1. **Discovery:** The moment we visit u, increment the `time` counter
2. **Update Vertex:** Set u's discovery time $d[u]$ to the new time, and colour u **Grey**
3. **Explore Paths:** For every Neighbour v in u's adjacency list:
   - If v is **White:**
     - Set its predecessor to be u $\pi[v] = u$
     - Recursively call `DFS_Visit(v)` to travel further down the branch
4. **Increment Time:** Once all neighbours have been fully explored (or if it is a dead end), increment the `time` counter again
5. **Final Vertex Update:** Set u's finish time $f[u]$ to the current time, and colour u **Black**

Code Implementation: DFS (Tested in VS Code)

```
# Define the basic Graph class
class Graph:
    def __init__(self, vertices, adjacency_list):
        self.vertices = vertices # List of Vertices
        self.adjacency_list = adjacency_list # Dictionary

# Let G be the Graph
def depth_first_search(G):
    # Set up with empty dictionaries
    colour = {}
    d = {}      # Discovery Times
    f = {}      # Finish Times
    pi = {}     # Predecessors

    # Initialise all vertices to White and predecessors to None
    for v in G.vertices:
        colour[v] = "WHITE"
        pi[v] = None

    # We use the 'nonlocal' keyword inside the nested function to access the
variable globally
    time = 0  # Set the global time counter to 0

    # --- Recursive Explorer Function (runs for each vertex u) ---
    def dfs_visit(u):
        # nonlocal keyword so python treats time as a global variable
        nonlocal time
```

```
        time += 1  # Discovery: Increment the time counter

        # Update Vertex (u) Info
        d[u] = time
        colour[u] = "GREY"

        # Explore the paths of each neighbour of u
        for v in G.adjacency_list[u]:
            if colour[v] == "WHITE":
                pi[v] = u       # Set parent to be u
                dfs_visit(v)    # Recursive call to go deeper into the path

        # Increment Time once each neighbour's path has been explored
        time += 1

        # Final Vertex Update for u
        f[u] = time          # Set u's finish time
        colour[u] = "BLACK"  # Colour u Black

    # Forest Loop: Iterate through every vertex u in the graph
    for u in G.vertices:
        # If u is White, call the recursive exploration function
DFS_Visit(u)
        if colour[u] == "WHITE":
            dfs_visit(u)

    return d, f, pi
```

Run Code

```
# ----------------------------------------------------------
# Works with the code above (copy-paste into VS code)
# ----------------------------------------------------------
if __name__ == "__main__":

    # Initialise G using the same graph for the BFS example
    example_vertices = [1, 2, 3, 4, 5]
    example_adj_list = {
        1: [2, 3],
        2: [2, 4],  # Includes the self-loop
        3: [4],
        4: [3, 5],  # 3 and 4 share a bidirectional edge
        5: []       # Sink node
    }

    G = Graph(example_vertices, example_adj_list)

    # Run the search
    discovery, finish, predecessors = depth_first_search(G)
```

```
    print("Discovery Times (d):")
    print(discovery)

    print("\nFinish Times (f):")
    print(finish)

    print("\nPredecessor (Parent) Nodes:")
    print(predecessors)
```

Time & Space Complexities

Time Complexity: $O(V + E)$

- **Vertices:** The main loop takes $O(V)$ time to check the initial state of every vertex
- **Edges:** The recursive explorer function is called once per vertex, and it scans the adjacency list of that vertex

Time Complexity Exam Note: $O(V^2)$

- **Common Mistake:** Assuming the nested loops (vertices $\times$ neighbours) automatically mean $O(V^2)$ time complexity

Why it is $O(V + E)$ (Adjacency List):

- When an adjacency list is used, the algorithm only iterates over edges that *actually exist*
- So across the entire search, the inner loop runs exactly E times in total

When it IS $O(V^2)$ (Adjacency Matrix):

- When an adjacency matrix is used, the algorithm must scan an entire row of V elements for every single vertex just to find the edges
- This forces $V \times V$ operations

Example: $O(V + E)$ or $O(V^2)$

1. Adjacency List Example: $O(V + E)$

```
Vertices   Neighbours (Edges)
   A    -> [B] -> [C]
   B    -> [D]
   C    -> [D]
   D    -> [ ]  (Empty)
```

- The algorithm looks at 4 vertices (V), and across all 4 vertices, it checks exactly 4 edges. Therefore, the total operations $(4 + 4 = 8)$ scales linearly as $O(V + E)$
- (Doesn't waste time checking edges that don't exist)

Adjacency Matrix: $O(V^2)$

```
      [A]  [B]  [C]  [D]  <-- Target Vertices
[A]    0    1    1    0
[B]    0    0    0    1
[C]    0    0    0    1
[D]    0    0    0    0
 ^
Source Vertices
```

- The algorithm is forced to check V rows against V columns $(4 \times 4 = 16)$, therefore the overall time complexity is $O(V^2)$

Space Complexity: $O(V)$

- **The Baseline:** The auxiliary dictionaries (`colour` , `d` , `f` , `pi`) mean we must always store $4V$ items. Therefore, the minimum space requirement is always $O(V)$, regardless of the graph's shape
- **Call Stack:** Whether its a 'good' or 'bad' scenario depends on the recursion call stack, which changes based on the graph's structure

Worst-case Scenario: (Linear/Path Graph)

- A longer call stack occurs in a path graph
- **Example:** For $A \to B \to C \to D \to E$ the call-stack would have up to 5 active, paused functions waiting in memory simultaneously
- **Total Space:** $O(V)$ (Dictionaries) + $O(V)$ (Call Stack) = $O(V)$

Best-case Scenario: (Star Graph)

- **Call Stack Space:** $O(1)$ (Only ever goes 2 levels deep in the best case)
- **Dictionary Space:** $O(V)$ (4 dictionaries means we have to store $4V$ items)
- **Total Space Complexity:** $O(V) + O(1) = O(V)$

DFS Edge Classification (Important for Exam)

- When DFS traverses a directed graph, it categorises every edge (u, v) it crosses into one of four distinct types

- You can identify edges by the **colour** of the destination vertex v when it is first encountered, or by comparing their **Timestamps** (d for discovery, f for finish)

1. Tree Edges (New vertices)

- The standard edges that actively discover new vertices and build the Depth-First Forest. An edge (u, v) is a tree edge if it is the first edge to discover vertex v
- **Identify by Colour:** Destination vertex v is **White**
- **Identify by Time:** Vertex v is completely enveloped by u's timeframe
  - $d[u] < d[v] < f[v] < f[u]$

2. Back Edges (Cycles)

- Edges that point back up to the tree to an ancestor vertex
- **Exam Note:** Finding a back edge guarantees that the graph contains a cycle
- **Identify by Colour:** Destination vertex v is **Grey**
- **Identify by Time:** Vertex u is completely enveloped by v's timeframe (because v is the older ancestor)
  - $d[v] < d[u] < f[u] < f[v]$

3. Forward Edges (Shortcuts)

- Edges that point forward down the tree to a descendant vertex that has already been fully explored by a previous path (newly found shortcut - ignoring weights)
- **Identify by Colour:** Destination vertex v is **Black**
- **Identify by Time:** Same calculation as a **Tree Edge**, but you know it is a forward edge because v was previously discovered
  - $d[u] < d[v] < f[v] < f[u]$

4. Cross Edges

- All other edges, they point between different branches of the same tree, or between different trees of the same forest (No ancestral relationship)
- **Identify by Colour:** Destination vertex v is also **Black**
- **Identify by Time:** Vertex v was completely processed and finished *before* u was even discovered
  - $d[v] < f[v] < d[u] < f[u]$

DFS Graph Properties:

DFS on Undirected Graphs:

- Undirected edges can only ever have **Tree Edges** or **Back Edges**

- Forward or cross edges mathematically cannot exist during undirected DFS traversal

DFS to Find Cycles:

- A graph contains a cycle ***if and only if*** a DFS traversal discovers a **Back Edge**

Articulation Points:

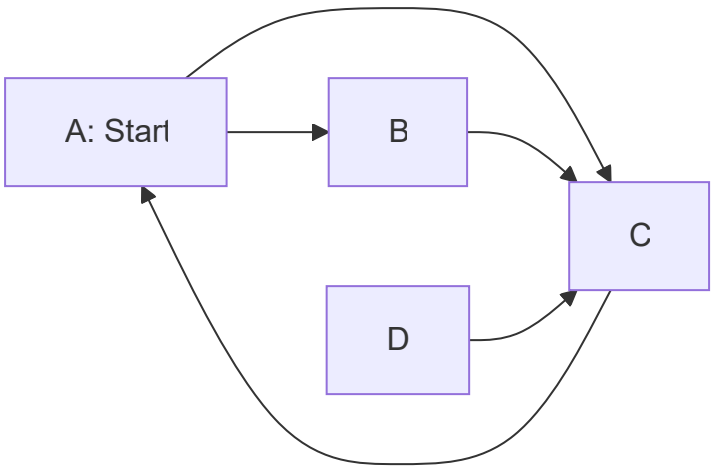
- An articulation point is a specific vertex whose removal entirely disconnects the graph (or increases the number of disconnected components). It acts as a critical "bridge" node holding different sections of the graph together

---

Example: DFS Edge Classification

Consider a directed graph with 4 vertices (A, B, C, D)

Diagram:



With the following adjacency list:

- **A:** [B, C]
- **B:** [C]
- **C:** [A]
- **D:** [C]

Steps (Timestamp Generation):

Use the above adjacency list to understand how Paths are chosen

1. **Visit A:** $d[A] = 1$ (Colour: Grey)
2. **Path** $A \to B$**:** B is White. $d[B] = 2$ (Colour: Grey)
3. **Path** $B \to C$**:** C is White. $d[C] = 3$ (Colour: Grey)
4. **Path** $C \to A$**:** A is Grey. (End of Branch)

5. **Finish C:** $f[C] = 4$ (Colour: Black). We backtrack to B
6. **Finish B:** $f[B] = 5$ (Colour: Black). We backtrack to A
7. **Path $A \to C$:** C is already Black
8. **Finish A:** $f[A] = 6$ (Colour: Black). The first tree is complete
9. **Main Loop:** Checks B (Black), Checks C (Black), Checks D (White)
10. **Visit D:** $d[D] = 7$ (Colour: Grey)
11. **Path $D \to C$:** C is already Black
12. **Finish D:** $f[D] = 8$ (Colour: Black). The forest is complete

Edge Classification:

Edge $A \to B$ (Tree Edge):

- **Colour:** B was White when discovered by A
- **Time:** $1 < 2 < 5 < 6$ $\big(d[A] < d[B] < f[B] < f[A]\big)$. B's life is entirely inside A's

Edge $B \to C$ (Tree Edge):

- **Reason:** C was White when discovered by B
- **Time:** $2 < 3 < 4 < 5$ $\big(d[B] < d[C] < f[C] < f[B]\big)$. C's life is entirely inside B's

Edge $C \to A$ (Back Edge):

- **Reason:** A was Grey when encountered. It points back up to an ancestor, proving the cycle A-B-C-A
- **Time:** $1 < 3 < 4 < 6$ $\big(d[A] < d[C] < f[C] < f[A]\big)$. C is enveloped by A's timeline, meaning A is the older ancestor

Edge $A \to C$ (Forward Edge):

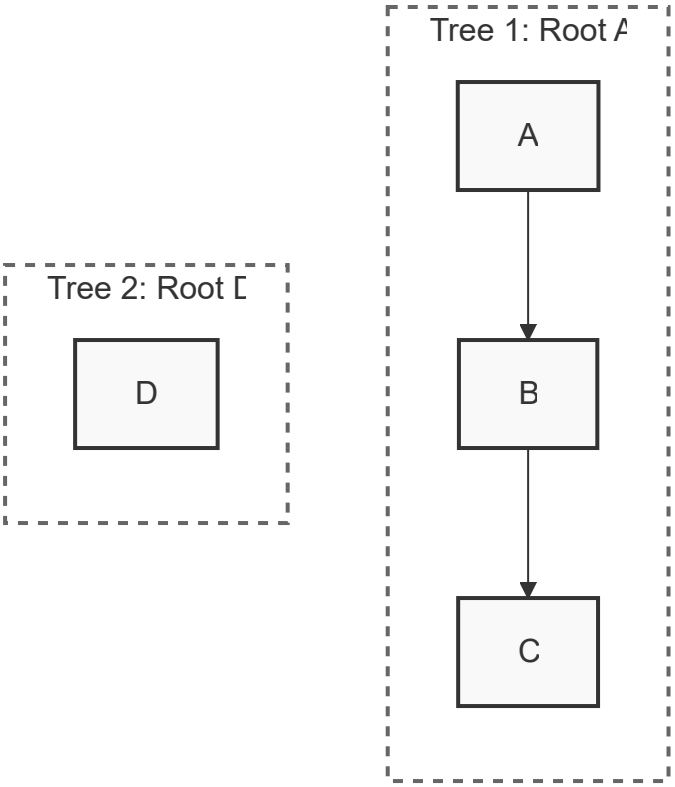
- **Reason:** C was Black when A checked it. A is the ancestor of C, but C was already fully explored via the A $\to$ B $\to$ C path. This edge is just a direct shortcut
- **Time:** $1 < 3 < 4 < 6$ $\big(d[A] < d[C] < f[C] < f[A]\big)$. The math matches a Tree edge, but because C was already Black, we know it is a Forward edge

Edge D $\to$ C (Cross Edge):

- **Reason:** C was Black when D checked it, but D is not an ancestor of C. They are in completely different branches of the timeline
- **Time:** $3 < 4 < 7 < 8$ $\big(d[C] < f[C] < d[D] < f[D]\big)$. C's entire life started and finished before D was even discovered

---

Resulting Depth-First Forest:



Related Pages:

- Intro to Graph Theory
- Shortest Paths, Cycles & Connectivity
- Trees & Isomorphism
- Breadth-First Search
- Minimum Spanning Trees